

Handwritten digits classification using neural network

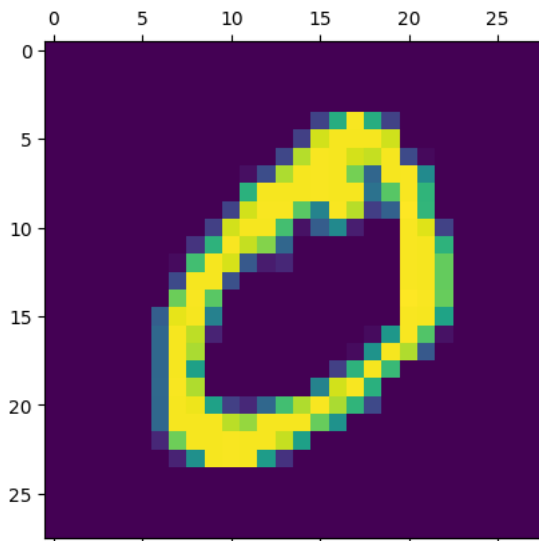
In this notebook we will classify handwritten digits using a simple neural network which has only input and output layers. We will then add a hidden layer and see how the performance of the model improves

```
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt
%matplotlib inline
import numpy as np
```

```
(X_train, y_train) , (X_test, y_test) = keras.datasets.mnist.load_data()
```

```
plt.matshow(X_train[1])
```

<matplotlib.image.AxesImage at 0x7a1fa8282630>



```
y_train[1]
```

```
np.uint8(0)
```

```
X_train = X_train / 255
X_test = X_test / 255
```

```
X_train[0]
```

```

0.0745098, 0.88027451, 0.99215686, 0.99215686, 0.99215686,
0.99215686, 0.95686275, 0.52156863, 0.04313725, 0.
0.
, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
],
[0.
, 0.
, 0.
, 0.
, 0.53333333,
0.99215686, 0.99215686, 0.99215686, 0.83137255, 0.52941176,
0.51764706, 0.0627451, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
],
[0.
, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
],
[0.
, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
],
[0.
, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
, 0.
, 0.
,
0.
, 0.
, 0.
]]))

```

```

X_train_flattened = X_train.reshape(len(X_train), 28*28)
X_test_flattened = X_test.reshape(len(X_test), 28*28)

```

```
X_train_flattened.shape
```

```
(60000, 784)
```

```
X_train_flattened[0]
```

[Show hidden output](#)

Very simple neural network with no hidden layers



```

model = keras.Sequential([
    keras.layers.Dense(10, input_shape=(784,), activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(X_train_flattened, y_train, epochs=5)

```

[Show hidden output](#)

```
model.evaluate(X_test_flattened, y_test)
```

[Show hidden output](#)

```

y_predicted = model.predict(X_test_flattened)
y_predicted[0]

```

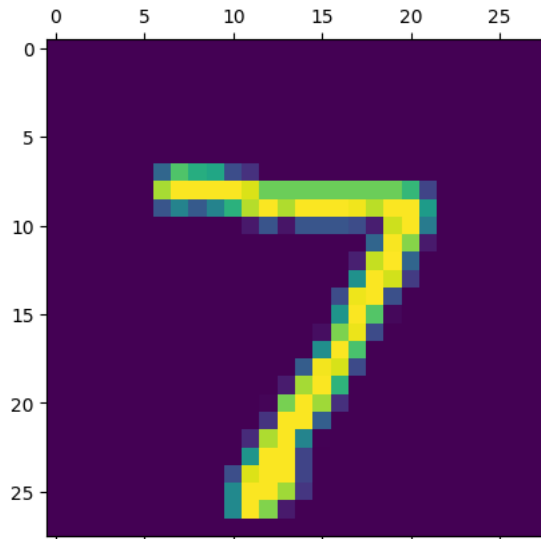
```

313/313 — 0s 1ms/step
array([2.3478705e-02, 3.8432350e-07, 4.2286959e-02, 9.5681876e-01,
       2.1715015e-03, 1.1584571e-01, 1.5780716e-06, 9.9979764e-01,
       8.2609363e-02, 6.9621879e-01], dtype=float32)

```

```
plt.matshow(X_test[0])
```

```
<matplotlib.image.AxesImage at 0x7a1fa8254c80>
```



np.argmax finds a maximum element from an array and returns the index of it

```
np.argmax(y_predicted[0])
```

```
np.int64(7)
```

```
y_predicted_labels = [np.argmax(i) for i in y_predicted]
```

```
y_predicted_labels[:5]
```

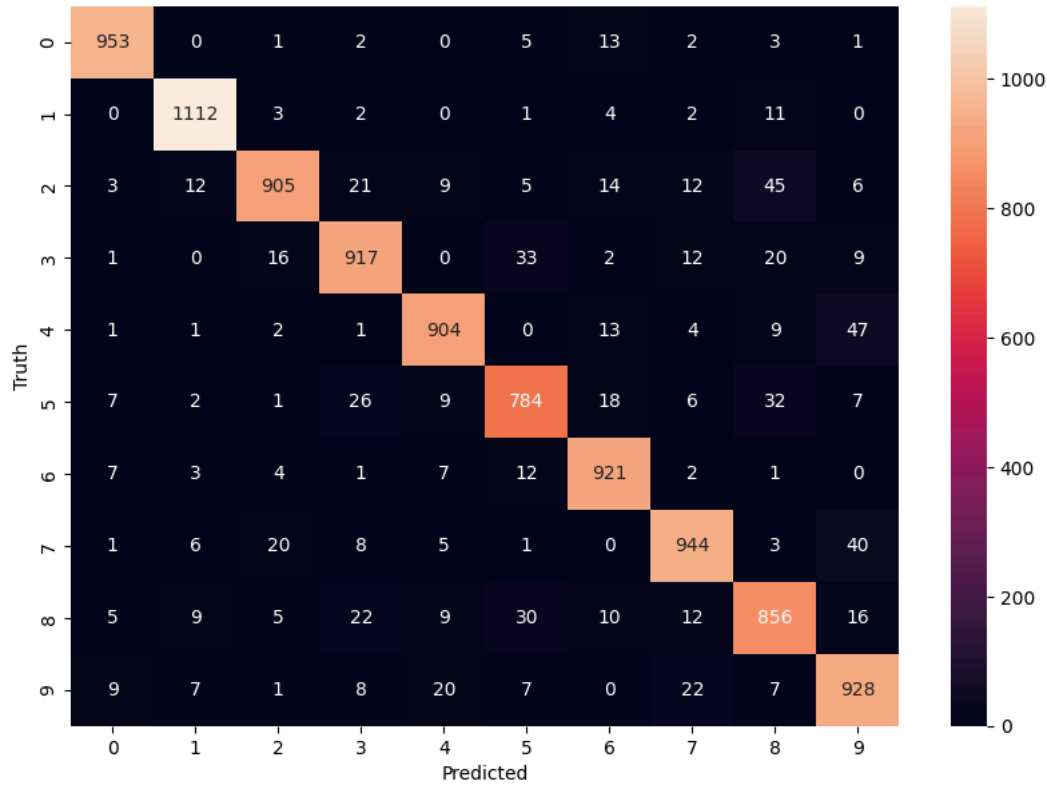
```
[np.int64(7), np.int64(2), np.int64(1), np.int64(0), np.int64(4)]
```

```
cm = tf.math.confusion_matrix(labels=y_test, predictions=y_predicted_labels)
cm
```

```
<tf.Tensor: shape=(10, 10), dtype=int32, numpy=
array([[ 953,    0,    1,    2,    0,    5,   13,    2,    3,    1],
       [    0, 1112,    3,    2,    0,    1,    4,    2,   11,    0],
       [    3,   12,   905,   21,    9,    5,   14,   12,   45,    6],
       [    1,    0,   16,   917,    0,   33,    2,   12,   20,    9],
       [    1,    1,    2,    1,   904,    0,   13,    4,    9,   47],
       [    7,    2,    1,   26,    9,   784,   18,    6,   32,    7],
       [    7,    3,    4,    1,    7,   12,   921,    2,    1,    0],
       [    1,    6,   20,    8,    5,    1,    0,   944,    3,   40],
       [    5,    9,    5,   22,    9,   30,   10,   12,   856,   16],
       [    9,    7,    1,    8,   20,    7,    0,   22,    7,   928]],
      dtype=int32)>
```

```
import seaborn as sn
plt.figure(figsize = (10,7))
sn.heatmap(cm, annot=True, fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Truth')
```

Text(95.7222222222221, 0.5, 'Truth')



Using hidden layer

```
model = keras.Sequential([
    keras.layers.Dense(100, input_shape=(784,), activation='relu'),
    keras.layers.Dense(10, activation='sigmoid')
])
```

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

```
model.fit(X_train_flattened, y_train, epochs=5)
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input\_shape` argument to `Dense` layers. It has no effect and will be removed in a future version.

```
Epoch 1/5
1875/1875 — 8s 4ms/step — accuracy: 0.9200 — loss: 0.2785
Epoch 2/5
1875/1875 — 7s 4ms/step — accuracy: 0.9625 — loss: 0.1272
Epoch 3/5
1875/1875 — 7s 4ms/step — accuracy: 0.9733 — loss: 0.0891
Epoch 4/5
1875/1875 — 6s 3ms/step — accuracy: 0.9799 — loss: 0.0670
Epoch 5/5
1875/1875 — 7s 4ms/step — accuracy: 0.9834 — loss: 0.0541
<keras.src.callbacks.history.History at 0x7a1fc64aeb70>
```

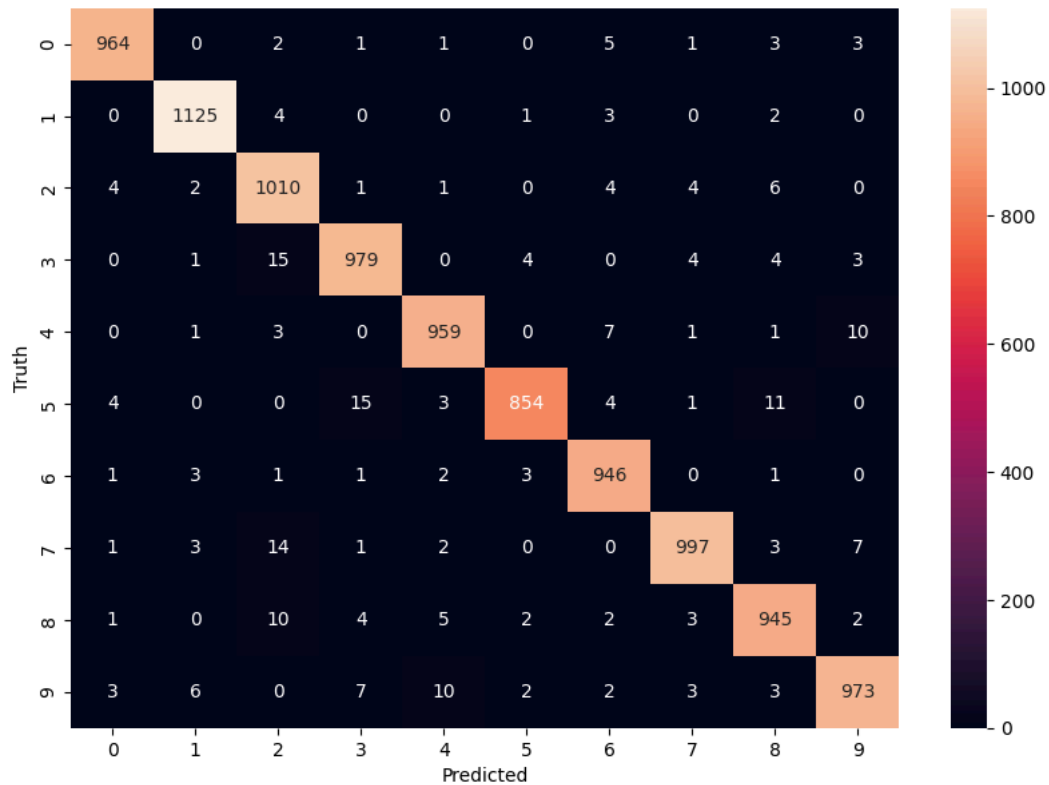
```
model.evaluate(X_test_flattened, y_test)
```

```
313/313 — 1s 2ms/step — accuracy: 0.9752 — loss: 0.0821
[0.08212970942258835, 0.9751999974250793]
```

```
y_predicted = model.predict(X_test_flattened)
y_predicted_labels = [np.argmax(i) for i in y_predicted]
cm = tf.math.confusion_matrix(labels=y_test, predictions=y_predicted_labels)
```

```
plt.figure(figsize = (10,7))
sn.heatmap(cm, annot=True, fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Truth')
```

313/313 ————— 1s 2ms/step
Text(95.7222222222221, 0.5, 'Truth')



Using Flatten layer so that we don't have to call .reshape on input dataset

```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(100, activation='relu'),
    keras.layers.Dense(10, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(X_train, y_train, epochs=10)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `in
super().__init__(**kwargs)
Epoch 1/10
1875/1875 ————— 8s 4ms/step - accuracy: 0.9216 - loss: 0.2780
Epoch 2/10
1875/1875 ————— 11s 4ms/step - accuracy: 0.9631 - loss: 0.1264
Epoch 3/10
1875/1875 ————— 7s 3ms/step - accuracy: 0.9738 - loss: 0.0891
Epoch 4/10
1875/1875 ————— 6s 3ms/step - accuracy: 0.9796 - loss: 0.0679
Epoch 5/10
1875/1875 ————— 7s 4ms/step - accuracy: 0.9837 - loss: 0.0520
Epoch 6/10
1875/1875 ————— 6s 3ms/step - accuracy: 0.9867 - loss: 0.0429
Epoch 7/10
1875/1875 ————— 7s 4ms/step - accuracy: 0.9892 - loss: 0.0349
Epoch 8/10
1875/1875 ————— 6s 3ms/step - accuracy: 0.9920 - loss: 0.0273
Epoch 9/10
1875/1875 ————— 7s 4ms/step - accuracy: 0.9923 - loss: 0.0239
Epoch 10/10
1875/1875 ————— 6s 3ms/step - accuracy: 0.9942 - loss: 0.0196
<keras.src.callbacks.history.History at 0x7a1fa84f7ef0>
```

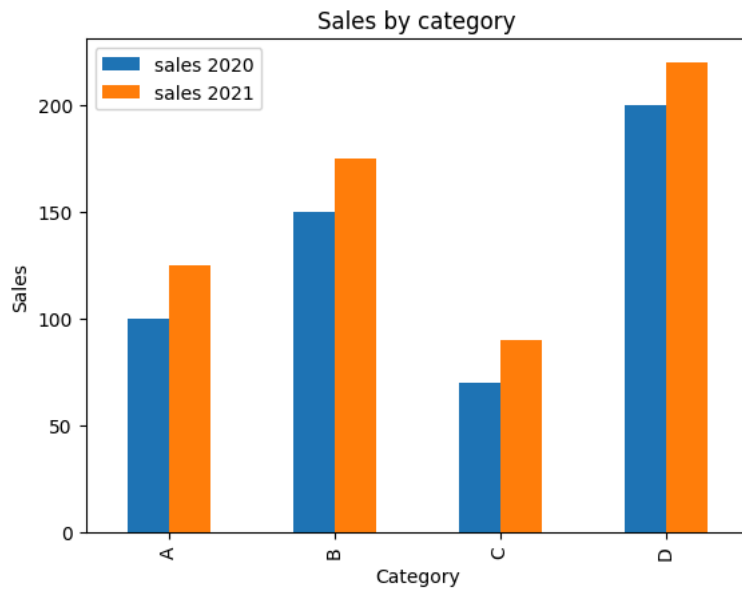
```
model.evaluate(X_test, y_test)
```

```
313/313 ————— 1s 2ms/step - accuracy: 0.9746 - loss: 0.0913
[0.09129904210567474, 0.9746000170707703]
```

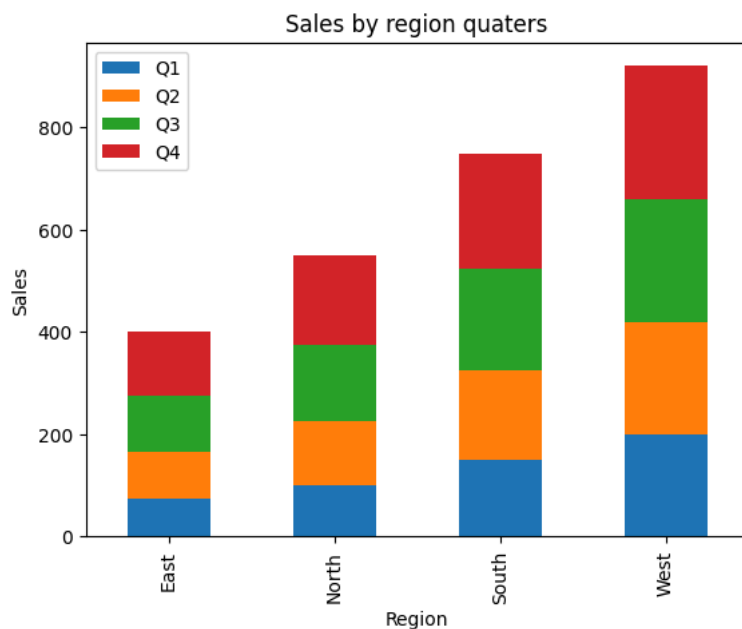
LAB 2

```
import pandas as pd
import matplotlib.pyplot as plt

data =pd.DataFrame({
    'category':['A','B','C','D'],
    'sales 2020':[100,150,70,200],
    'sales 2021':[125,175,90,220]
})
data.plot(x='category',kind='bar')
plt.xlabel('Category')
plt.ylabel('Sales')
plt.title('Sales by category')
plt.show()
```



```
import pandas as pd
import matplotlib.pyplot as plt
data =pd.DataFrame({
    'region':['North','South','East','West'],
    'Q1':[100,150,75,200],
    'Q2':[125,175,90,220],
    'Q3':[150,200,110,240],
    'Q4':[175,225,125,260]
})
data =data.sort_values(by='Q4')
data.plot(x='region',kind='bar',stacked=True)
plt.xlabel('Region')
plt.ylabel('Sales')
plt.title('Sales by region quaters')
plt.show()
```



```
import pandas as pd
import matplotlib.pyplot as plt

data=pd.DataFrame({
    'Age': [25,31,38,42,57,61,65,71],
    'Income': [34000,44000,52000,61000,74000,88000,98000,100000]
})
plt.scatter(data['Age'],data['Income'])
plt.xlabel('Age')
plt.ylabel('Income')
plt.title('Age vs Income')
plt.show()
```

